

Chapter 4: Local Descent

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

MIT Press

Chapter Outline

- 1 4.1 Descent Direction Iteration
- 2 4.2 Step Factors
- 3 4.3 Line Search
- 4 4.4 Approximate Line Search — Wolfe Conditions
- 5 4.5 Trust Region Methods
- 6 4.6 Termination Conditions
- 7 4.7 Summary & Exercises

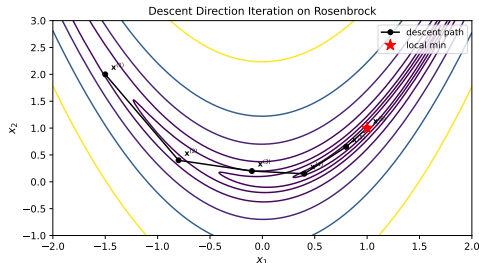
Motivation: Multivariate Optimization

Goal: minimize $f : \mathbb{R}^n \rightarrow \mathbb{R}$

- May be non-convex, may have many local minima
- Exact methods (Chapter 3) do not scale to $n \gg 1$
- **Key idea:** iteratively move the design point in a direction that *decreases* f

Descent direction iteration:

- 1 Check local model; if already at a minimum, stop.
- 2 Choose a descent direction $\mathbf{d}^{(k)}$ from the current point $\mathbf{x}^{(k)}$.
- 3 Choose a step factor $\alpha^{(k)} > 0$.
- 4 Update: $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$.



What Is a Descent Direction?

Definition: Descent Direction

A vector $\mathbf{d} \in \mathbb{R}^n$ is a *descent direction* at $\mathbf{x}$ if

$$\nabla_{\mathbf{d}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{d} < 0.$$

The *directional derivative* $\nabla_{\mathbf{d}} f$ is negative, so moving a small positive distance along $\mathbf{d}$ decreases f .

Properties:

- The negative gradient $-\nabla f(\mathbf{x})$ is always a descent direction (steepest descent).
- Newton direction, quasi-Newton directions are also descent directions under suitable conditions.
- Momentum directions (e.g. in machine learning) can provide descent with curvature information.

Iterative Update Rule

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$$

$\mathbf{x}^{(k)}$ current design point

$\mathbf{d}^{(k)}$ descent direction

$\alpha^{(k)}$ step factor (step size)

Algorithm 4.1: Descent Direction Iteration (Python)

```
1 import numpy as np
2
3 def descent_direction_iteration(f, grad_f, x0, direction_fn,
4                               step_fn, max_iter=1000, tol=1e-6):
5     """
6     Generic descent direction iteration.
7     Args:
8         f           : objective function  f(x) -> float
9         grad_f      : gradient function   grad_f(x) -> ndarray
10        x0           : initial design point (ndarray)
11        direction_fn: d = direction_fn(x, g) -> descent direction
12        step_fn      : alpha = step_fn(f, grad_f, x, d) -> step factor
13    """
14    x = x0.copy().astype(float)
15    history = [x.copy()]
16    for k in range(max_iter):
17        g = grad_f(x)
18        if np.linalg.norm(g) < tol:
19            break
20        d = direction_fn(x, g) # choose descent direction
21        alpha = step_fn(f, grad_f, x, d) # choose step factor
22        x = x + alpha * d
23        history.append(x.copy())
24    return x, history
```

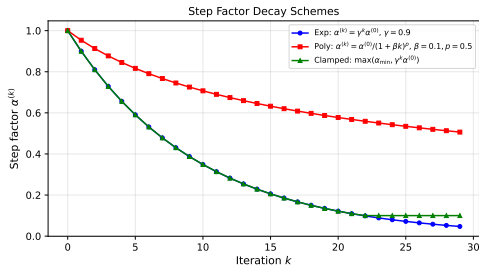
Step Factors (Learning Rate)

The *step factor* (also called *step size* or *learning rate*) $\alpha^{(k)}$ controls how far to move.

Fixed step factor: $\alpha^{(k)} = \alpha_0$ for all k .

- Simple, but must be chosen carefully.
- Too large $\Rightarrow$ divergence.
- Too small $\Rightarrow$ slow convergence.

Decaying step factors allow large early steps (exploration) and small later steps (fine-tuning).



Step Factor Decay Schemes

Exponential Decay (Eq. 4.2)

$$\alpha^{(k)} = \gamma^k \alpha^{(0)} = \gamma \alpha^{(k-1)}, \quad 0 < \gamma < 1$$

Total distance bounded: $\sum_{k=0}^{\infty} \delta \alpha^{(0)} \gamma^k = \frac{\delta \alpha^{(0)}}{1 - \gamma}$ (if $\|\mathbf{d}^{(k)}\| \leq \delta$)

Polynomial Decay (Eq. 4.4)

$$\alpha^{(k)} = \frac{\alpha^{(0)}}{(1 + \beta k)^p}, \quad \beta > 0, p > 0$$

Unbounded total distance when $0 < p \leq 1$.

Clamped Exponential Decay (Eq. 4.5)

$$\alpha^{(k)} = \max(\alpha_{\min}, \gamma^k \alpha^{(0)})$$

Prevents premature stagnation; fast early decay, stable later.

Step Factor Decay — Python Example

```
1 import numpy as np
2
3 def exponential_decay(alpha0, gamma, k):
4     """alpha^(k) = gamma^k * alpha0, 0 < gamma < 1"""
5     return gamma**k * alpha0
6
7 def polynomial_decay(alpha0, beta, p, k):
8     """alpha^(k) = alpha0 / (1 + beta*k)^p, beta>0, p>0"""
9     return alpha0 / (1.0 + beta * k)**p
10
11 def clamped_decay(alpha0, gamma, alpha_min, k):
12     """alpha^(k) = max(alpha_min, gamma^k * alpha0)"""
13     return max(alpha_min, gamma**k * alpha0)
14
15 # Example: gradient descent on f(x) = x^2 with decaying lr
16 def f(x): return x**2
17 def df(x): return 2*x
18
19 x = 3.0
20 for k in range(20):
21     alpha = exponential_decay(alpha0=1.0, gamma=0.85, k=k)
22     x = x - alpha * df(x)
23     if abs(df(x)) < 1e-6:
24         break
25 print(f"Converged to x={x:.6f}, f(x)={f(x):.2e}")
```


Line Search: Optimizing Along a Direction

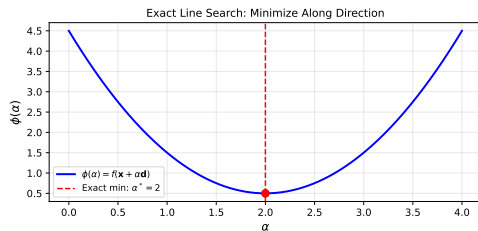
Exact line search: choose α that *minimizes* f along the direction $\mathbf{d}$:

$$\alpha^* = \arg \min_{\alpha \geq 0} f(\mathbf{x} + \alpha \mathbf{d})$$

- Reduces to 1-D optimization (Chapter 3 methods).
- Expensive per iteration if f is costly.
- Quadratic f : closed-form solution exists.

Quadratic example: For $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x} - \mathbf{g}^\top \mathbf{x}$, steepest descent direction $\mathbf{d} = -\nabla f$:

$$\alpha^* = \frac{\mathbf{d}^\top \mathbf{d}}{\mathbf{d}^\top \mathbf{H} \mathbf{d}}$$



Exact Line Search — Python (SciPy)

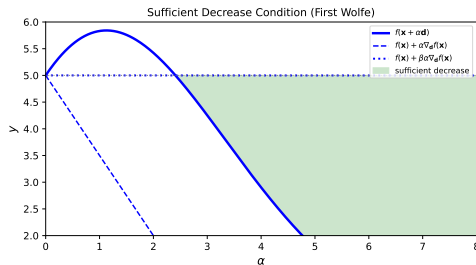
```
1  import numpy as np
2  from scipy.optimize import minimize_scalar
3
4  def exact_line_search(f, x, d):
5      """
6      Minimize  $f(x + \alpha * d)$  over  $\alpha \geq 0$ .
7      Returns optimal step size  $\alpha$ *.
8      """
9      phi = lambda alpha: f(x + alpha * d)
10     result = minimize_scalar(phi, bounds=(0, 10), method='bounded')
11     return result.x
12
13     # Example:  $f(x_1, x_2) = x_1^2 + x_1 * x_2 + x_2^2$ 
14     def f(x):
15         return x[0]**2 + x[0]*x[1] + x[1]**2
16
17     def grad_f(x):
18         return np.array([2*x[0] + x[1], x[0] + 2*x[1]])
19
20     x = np.array([3.0, 2.0])
21     d = -grad_f(x) # steepest descent direction
22     alpha_star = exact_line_search(f, x, d)
23     x_new = x + alpha_star * d
24
25     print(f"alpha* = {alpha_star:.4f}")
26     print(f"x_new = {x_new}")
27     print(f"f(x_new) = {f(x_new):.4f}")
```

Backtracking Line Search

Idea: Start with a large α and reduce it until the *sufficient decrease condition* (Armijo, first Wolfe) is met:

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla f(\mathbf{x})^\top \mathbf{d}$$

- $\beta \in (0, 1)$, typically 10^{-4} .
- Each iteration: $\alpha \leftarrow p \alpha$, $p \in (0, 1)$, typically $p = 0.5$.
- Guaranteed to terminate if $\mathbf{d}$ is a descent direction.
- Much cheaper than exact line search.



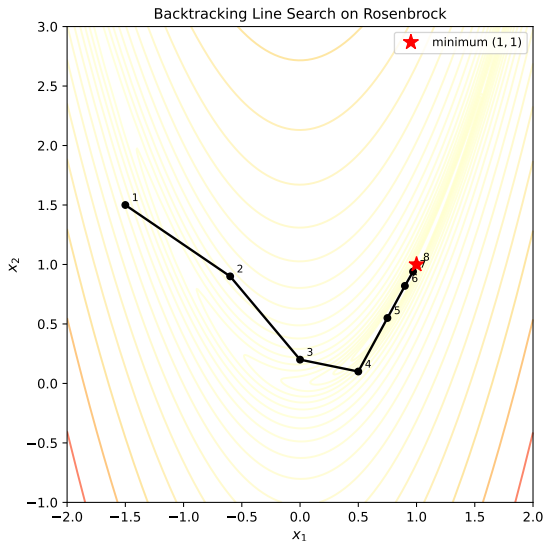
Backtracking on Rosenbrock

Algorithm 4.3: Backtracking Line Search

- 1 $y, \mathbf{g} \leftarrow f(\mathbf{x}), \nabla f(\mathbf{x})$
- 2 **while** $f(\mathbf{x} + \alpha \mathbf{d}) > y + \beta \alpha (\mathbf{g}^\top \mathbf{d})$:
 - $\alpha \leftarrow p \alpha$
- 3 **return** α

Parameters:

α	initial step (max step)
p	reduction factor (0.5)
β	Armijo parameter (10^{-4})



Backtracking Line Search — Python

```
1  import numpy as np
2
3  def backtracking_line_search(f, grad_f, x, d, alpha=1.0,
4                               p=0.5, beta=1e-4):
5      """
6      Backtracking line search (Armijo condition).
7      Reduces alpha by factor p until sufficient decrease is met.
8      """
9      y = f(x)
10     g = grad_f(x)
11     slope = g @ d                # directional derivative (must be < 0)
12     while f(x + alpha * d) > y + beta * alpha * slope:
13         alpha *= p
14     return alpha
15
16     # Numerical example (Example 4.2 from book)
17     # f(x1,x2) = x1^2 + x1*x2 + x2^2, x=[1,2], d=[-1,-1]
18     def f(x): return x[0]**2 + x[0]*x[1] + x[1]**2
19     def grad_f(x): return np.array([2*x[0]+x[1], x[0]+2*x[1]])
20
21     x = np.array([1.0, 2.0])    # f(x) = 1 + 2 + 4 = 7
22     d = np.array([-1.0, -1.0]) # descent direction
23     g = grad_f(x)              # g = [4, 5]
24
25     # Iteration 1: alpha=10 -> f([1,2]+10[-1,-1]) = f([-9,-8]) = 217 > 6.991
26     # Iteration 2: alpha=5 -> f([-4,-3]) = 37 > 6.996
27     # Iteration 3: alpha=2.5 -> f([-1.5,-0.5]) = 3.25 <= 6.998 *accept*
```

Why Approximate Line Search?

- Exact line search is too costly in practice.
- **Backtracking** only enforces *sufficient decrease* — it may accept very small steps.
- We want conditions that guarantee the step is:
 - 1 Not too small (*sufficient decrease*).
 - 2 Not too large (*curvature condition*).

Wolfe Conditions (sufficient decrease + curvature)

First Wolfe (Armijo):

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla f(\mathbf{x})^\top \mathbf{d}$$

Second Wolfe (curvature):

$$\nabla f(\mathbf{x} + \alpha \mathbf{d})^\top \mathbf{d} \geq \sigma \nabla f(\mathbf{x})^\top \mathbf{d}$$

with $0 < \beta < \sigma < 1$; typically $\beta = 10^{-4}$, $\sigma = 0.9$.

Strong Wolfe Conditions

The *strong Wolfe conditions* add an absolute value to the curvature:

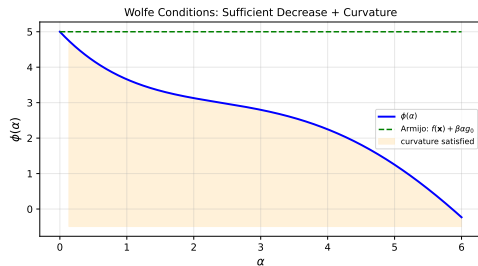
❶ **Sufficient decrease:**

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla f(\mathbf{x})^\top \mathbf{d}$$

❷ **Strong curvature:** $|\nabla f(\mathbf{x} + \alpha \mathbf{d})^\top \mathbf{d}| \leq \sigma |\nabla f(\mathbf{x})^\top \mathbf{d}|$

Why strong?

- Prevents steps near a local max of $\phi(\alpha)$.
- Required by conjugate gradient and quasi-Newton methods.
- σ close to 1: loose curvature (backtracking-like).
- σ close to β : tight, near-exact line search.



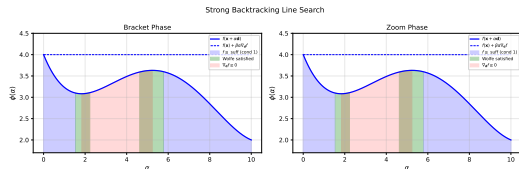
Strong Backtracking Algorithm

Algorithm 4.4: Strong Backtracking

Bracket phase: Find an interval $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ guaranteed to contain a strong-Wolfe step:

- If $f(\mathbf{x} + \alpha \mathbf{d}) > f(\mathbf{x}) + \beta \alpha g_0$ or $f(\mathbf{x} + \alpha \mathbf{d}) \geq f_{\text{prev}}$: bracket $[\alpha_{\text{prev}}, \alpha]$, go to zoom.
- Compute $g = \nabla f(\mathbf{x} + \alpha \mathbf{d})^\top \mathbf{d}$. If $|g| \leq -\sigma g_0$: **return** α .
- If $g \geq 0$: bracket $[\alpha, \alpha_{\text{prev}}]$, zoom.
- Else: $\alpha_{\text{prev}} \leftarrow \alpha$, $\alpha \leftarrow 2\alpha$.

Zoom phase: Bisect $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ until strong Wolfe conditions are satisfied.



Strong Backtracking — Python

```
1 import numpy as np
2
3 def strong_backtracking(f, grad_f, x, d, alpha=1.0, beta=1e-4, sigma=0.1):
4     """Strong backtracking line search satisfying strong Wolfe conditions."""
5     y0 = f(x)
6     g0 = grad_f(x) @ d           # directional derivative at alpha=0
7     y_prev, alpha_prev = float('nan'), 0.0
8     alpha_lo = alpha_hi = float('nan')
9
10    # Bracket phase
11    while True:
12        y = f(x + alpha * d)
13        if y > y0 + beta * alpha * g0 or (not np.isnan(y_prev) and y >= y_prev):
14            alpha_lo, alpha_hi = alpha_prev, alpha
15            break
16        g = grad_f(x + alpha * d) @ d
17        if abs(g) <= -sigma * g0:
18            return alpha           # strong Wolfe satisfied
19        if g >= 0:
20            alpha_lo, alpha_hi = alpha, alpha_prev
21            break
22        y_prev, alpha_prev, alpha = y, alpha, 2 * alpha
23
24    # Zoom phase (bisection)
25    y_lo = f(x + alpha_lo * d)
26    while True:
27        alpha = (alpha_lo + alpha_hi) / 2.0
```

Wolfe Conditions: Bracketing Guarantee

An interval $[0, a]$ is **guaranteed to contain** a step satisfying the strong Wolfe conditions when *any* of the three bracketing conditions holds:

- ❶ $f(\mathbf{x} + \alpha \mathbf{d}) \geq f(\mathbf{x})$ (overshot)
- ❷ $f(\mathbf{x} + \alpha \mathbf{d}) > f(\mathbf{x}) + \beta \alpha \nabla_{\mathbf{d}} f(\mathbf{x})$ (violated Armijo)
- ❸ $\nabla_{\mathbf{d}} f(\mathbf{x} + \alpha \mathbf{d}) \geq 0$ (gradient changed sign)

The zoom phase narrows the interval by bisection, preserving the bracket invariant.

Condition Numbers (Eq. 4.11–4.13)

$$f(\mathbf{x} + \alpha \mathbf{d}) \geq f(\mathbf{x})$$

$$f(\mathbf{x} + \alpha \mathbf{d}) > f(\mathbf{x}) + \beta \alpha \nabla_{\mathbf{d}} f(\mathbf{x})$$

$$\nabla_{\mathbf{d}} f(\mathbf{x} + \alpha \mathbf{d}) \geq 0$$

Numerical Example: Backtracking (Example 4.2)

Setup

$$f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2, \quad \mathbf{x} = [1, 2]^\top, \quad \mathbf{d} = [-1, -1]^\top, \quad \alpha_{\max} = 10, \quad p = 0.5, \quad \beta = 10^{-4}.$$

- $f(\mathbf{x}) = 1 + 2 + 4 = 7, \quad \mathbf{g} = \nabla f(\mathbf{x}) = [4, 5]^\top$

Iteration 1: $\alpha = 10$

$$f([1, 2] + 10[-1, -1]) = f([-9, -8]) = 217$$

$$7 + 10^{-4} \cdot 10 \cdot [4, 5]^\top [-1, -1] = 6.991$$

$$217 \leq 6.991 \Rightarrow \text{FAIL}$$

Iteration 2: $\alpha = 5$

$$f([-4, -3]) = 37$$

$$37 \leq 6.996 \Rightarrow \text{FAIL}$$

Iteration 3: $\alpha = 2.5$

$$f([-1.5, -0.5]) = 3.25$$

$$3.25 \leq 6.998 \Rightarrow \text{ACCEPT}$$

Final step: $\alpha^* = 2.5,$

$$\mathbf{x}_{\text{new}} = [-1.5, -0.5]$$

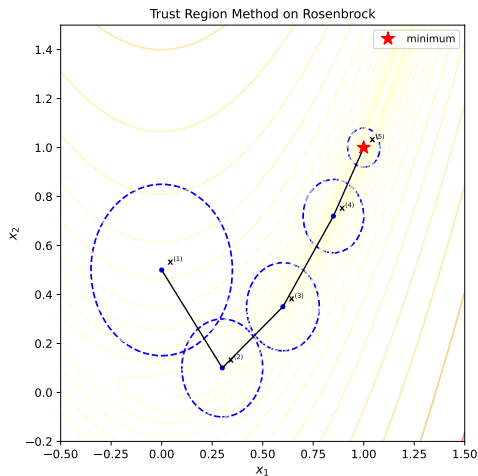
Trust Region: Concept

Idea: Instead of choosing a *direction* and then a *step*, simultaneously choose the step within a trusted region.

- Local quadratic model:

$$\hat{f}(\mathbf{x}') \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{x}' - \mathbf{x}) + \frac{1}{2}(\mathbf{x}' - \mathbf{x})^\top \mathbf{H}(\mathbf{x}' - \mathbf{x})$$

- Solve: minimize $\hat{f}$ subject to $\|\mathbf{x}' - \mathbf{x}\| \leq \delta$
- Trust region radius δ is adapted based on how well the model predicts the actual decrease.



Trust Region: Improvement Ratio

Improvement Ratio (Eq. 4.15)

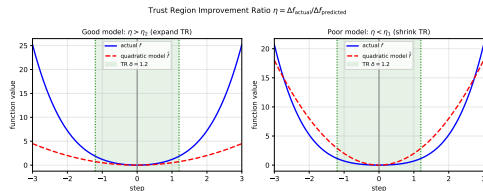
$$\eta = \frac{f(\mathbf{x}) - f(\mathbf{x}')}{f(\mathbf{x}) - \hat{f}(\mathbf{x}')} = \frac{\Delta f_{\text{actual}}}{\Delta f_{\text{predicted}}}$$

$\mathbf{x}'$ candidate next point
 $\hat{f}(\mathbf{x}')$ quadratic model prediction

Trust region update rules:

- $\eta < \eta_1$ (bad model): shrink $\delta \leftarrow \gamma_1 \delta$
- $\eta_1 \leq \eta \leq \eta_2$: keep δ
- $\eta > \eta_2$ (good model): expand $\delta \leftarrow \gamma_2 \delta$

Typical: $\eta_1 = 0.25$, $\eta_2 = 0.75$, $\gamma_1 = 0.5$, $\gamma_2 = 2$.



Trust Region Subproblem

Circular Trust Region Subproblem (Eq. 4.14)

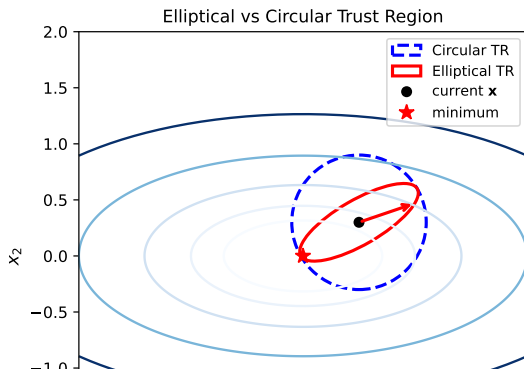
$$\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}'} \left[f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)})^\top (\mathbf{x}' - \mathbf{x}^{(k)}) + \frac{1}{2} (\mathbf{x}' - \mathbf{x}^{(k)})^\top \mathbf{H}(\mathbf{x}' - \mathbf{x}^{(k)}) \right]$$

subject to $\|\mathbf{x}' - \mathbf{x}^{(k)}\| \leq \delta$

Elliptical trust region (Eq. 4.16):

$$\|\mathbf{x} - \mathbf{x}_0\|_{\mathbf{E}}^2 = (\mathbf{x} - \mathbf{x}_0)^\top \mathbf{E} (\mathbf{x} - \mathbf{x}_0) \leq 1$$

The matrix $\mathbf{E}$ elongates the region along directions of higher confidence; adapts with each descent step.



Trust Region Descent — Python (SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def trust_region_step(grad, hess, x, delta):
5     """Solve the trust region subproblem (circular)."""
6     n = len(x)
7     def model(s):
8         return grad @ s + 0.5 * s @ hess @ s
9     def model_grad(s):
10        return grad + hess @ s
11    constraints = {'type': 'ineq',
12                  'fun': lambda s: delta - np.linalg.norm(s)}
13    s0 = np.zeros(n)
14    res = minimize(model, s0, jac=model_grad,
15                  constraints=constraints, method='SLSQP')
16    return res.x
17
18 def trust_region_descent(f, grad_f, hess_f, x0, delta=1.0,
19                          eta1=0.25, eta2=0.75,
20                          gamma1=0.5, gamma2=2.0, max_iter=100):
21    x = x0.copy().astype(float)
22    for _ in range(max_iter):
23        g = grad_f(x); H = hess_f(x)
24        s = trust_region_step(g, H, x, delta)
25        rho = (f(x) - f(x+s)) / (-(g @ s + 0.5 * s @ H @ s))
26        if rho > eta1:
27            x = x + s                # accept step
```

Termination Conditions

When should we stop iterating? Four common criteria:

- 1 **Maximum iterations (Eq. 4.17):**

$$k > k_{\max}$$

- 2 **Absolute improvement (Eq. 4.18):**

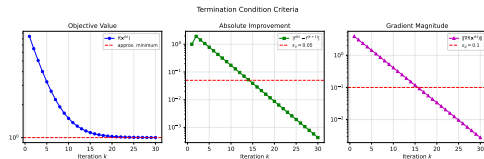
$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_a$$

- 3 **Relative improvement (Eq. 4.19):**

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_r \left| f(\mathbf{x}^{(k)}) \right|$$

- 4 **Gradient magnitude (Eq. 4.20):**

$$\|\nabla f(\mathbf{x}^{(k+1)})\| < \epsilon_g$$



Why Use Multiple Termination Conditions?

Single condition is fragile:

- *Max iterations only*: may stop too early or too late.
- *Gradient only*: may not converge if f is flat.
- *Abs. improvement only*: may stall on slow but non-zero progress.

Best practice: combine at least two:

- Gradient + max iterations (safety net).
- Add relative improvement for scale-invariance.

Random restarts: if multiple local minima are likely, restart from randomly chosen initial points after each convergence.

Typical Tolerances

Condition	Symbol	Typical value
Max iters	$k_{\max}$	10^3 – 10^5
Abs. improv.	ϵ_a	10^{-8}
Rel. improv.	ϵ_r	10^{-6}
Gradient	ϵ_g	10^{-6}

Termination Conditions — Python

```
1 import numpy as np
2
3 def check_termination(x_prev, x_curr, f_prev, f_curr, grad_curr,
4                       k, k_max=10000, eps_a=1e-8,
5                       eps_r=1e-6, eps_g=1e-6):
6     """
7     Returns (terminated: bool, reason: str).
8     Checks all four standard termination conditions.
9     """
10    if k >= k_max:
11        return True, f"max iterations ({k_max})"
12
13    abs_improv = f_prev - f_curr
14    if abs_improv < eps_a:
15        return True, f"absolute improvement {abs_improv:.2e} < {eps_a}"
16
17    rel_improv = abs_improv / (abs(f_prev) + 1e-12)
18    if rel_improv < eps_r:
19        return True, f"relative improvement {rel_improv:.2e} < {eps_r}"
20
21    grad_norm = np.linalg.norm(grad_curr)
22    if grad_norm < eps_g:
23        return True, f"gradient norm {grad_norm:.2e} < {eps_g}"
24
25    return False, "continuing"
```

Chapter 4 Summary

- **Descent direction iteration** provides a general framework: choose direction $\mathbf{d}^{(k)}$ and step $\alpha^{(k)}$, update $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{d}^{(k)}$.
- **Step factors** can be fixed, exponentially decayed, polynomially decayed, or chosen by line search.
- **Exact line search** minimizes f exactly along $\mathbf{d}$ (costly); **backtracking** is a cheap alternative.
- **Approximate line search** (Wolfe conditions) balances cost and quality:
 - First Wolfe (Armijo / sufficient decrease)
 - Second Wolfe (curvature condition)
 - Strong Wolfe: $|\nabla_{\mathbf{d}}f(\mathbf{x} + \alpha\mathbf{d})| \leq \sigma|\nabla_{\mathbf{d}}f(\mathbf{x})|$
- **Trust region methods** replace line search with a subproblem constrained to a trust ball of radius δ ; δ adapts via the improvement ratio η .
- **Termination conditions:** use multiple criteria (max iterations, absolute/relative improvement, gradient magnitude).

Key Formulas at a Glance

Descent Update

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$$

Step Decay

$$\alpha^{(k)} = \gamma^k \alpha^{(0)} \quad (0 < \gamma < 1)$$

First Wolfe (Armijo)

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla_{\mathbf{d}} f(\mathbf{x})$$

Second Wolfe (curvature)

$$\nabla_{\mathbf{d}} f(\mathbf{x} + \alpha \mathbf{d}) \geq \sigma \nabla_{\mathbf{d}} f(\mathbf{x})$$

Trust Region Subproblem

$$\min_{\|\mathbf{x}' - \mathbf{x}\| \leq \delta} \hat{f}(\mathbf{x}')$$

Improvement Ratio

$$\eta = \frac{f(\mathbf{x}) - f(\mathbf{x}')}{f(\mathbf{x}) - \hat{f}(\mathbf{x}')}$$

Termination

$$\begin{aligned} \|\nabla f(\mathbf{x}^{(k+1)})\| &< \epsilon_g \\ f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) &< \epsilon_a \end{aligned}$$

Exercises

Exercise 4.1

Why is it important to have more than one termination condition? Give a specific scenario where relying on *only* the gradient magnitude condition $\|\nabla f\| < \epsilon_g$ would be insufficient.

Exercise 4.2

Consider descent applied to $f(x) = \sin(x)$ starting at $x^{(1)} = -2$. What is the maximum step size such that f still decreases at each iteration (with the negative gradient direction)? Use the first Wolfe condition with $\beta = 0.1$.

Exercise 4.3 (second-order methods preview)

For $f(x_1, x_2) = x_1^2 + 2x_2^2$, starting at $\mathbf{x} = [1, 1]^\top$, show that the steepest descent direction is $[-2, -4]^\top$ and find the exact line search step size α^* .

Further Reading

- J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006. (Convergence analysis of descent methods; trust-region methods.)
- A. Conn, N. Gould, and P. Toint, *Trust Region Methods*, SIAM, 2000. (Definitive reference for trust-region algorithms.)
- SciPy documentation: `scipy.optimize.line_search_wolfe2` (Strong Wolfe line search), `scipy.optimize.minimize` with `method='trust-ncg'`.

End of Chapter 4